Construcción del “Hola Mundo” de un Backend para tu aplicación con NestJS, PostgreSQL y Prisma

---

Requerimientos

- **Node.js v24.14.1**
- **Npm 11.11.0**
- **NestJS 11.0.21**
- **PostgreSQL**
- **Docker**
- **Prisma 7.8.0**

Tecnologías utilizadas

El Backend de la aplicación a desarrollar utiliza:

- **NestJS:** Framework backend.
- **PostgreSQL:** Motor de base de datos.
- **Prisma ORM:** Comunicación entre la aplicación y la base de datos.
- **Docker**: Ejecución de PostgreSQL de forma local.

---

1. Instalación

Se utilizará TypeScript como lenguaje principal del backend, un lenguaje basado en JavaScript. Sobre este lenguaje se utilizará NestJS, un framework backend construido sobre Node.js que permite desarrollar APIs REST de forma modular, escalable y organizada mediante controladores, servicios y módulos. El proyecto utilizará PostgreSQL como sistema de base de datos con Docker que lo ejecuta en un contenedor, para simplificar la configuración y el despliegue del entorno de desarrollo, además utilizando Prisma, una herramienta que permite la comunicación entre el backend y la base de datos.

Node.js - NestJS CLI - PrismaORM

Se ejecutan los siguientes comandos en el CMD o terminal con el fin de descargar y revisar las respectivas instalaciones:

```bash
# Instalar Node.js
choco install nodejs
```

```bash
# Verificar Node.js y npm
node -v
npm -v

# Instalar NestJS CLI
npm install -g @nestjs/cli

# Verificar NestJS
nest -v

# Instalar Prisma en el proyecto
npm install prisma --save-dev
npm install @prisma/client
```

Docker – PostgreSQL

En este caso instalaremos Docker desktop (interfaz gráfica) para después instalar la imagen de PostgreSQL (una imagen en Docker es un paquete preconfigurado que contiene todo lo necesario para ejecutar un software dentro de un contenedor). Docker se suele instalar desde su página web oficial, a continuación, se encuentra el enlace para su descarga:

- Docker desktop: https://docs.docker.com/desktop/setup/install/windows-install/?utm_source=chatgpt.com

Posteriormente si bien se puede descargar la imagen desde la interfaz gráfica, insertando PostgreSQL en su buscador y presionando los botones “pull” y luego “run”, también es posible hacerlo desde consola, levantando además el contenedor en el proceso (se verá un poco más adelante)

2. Creación del proyecto backend

Se crea un directorio o carpeta nueva para el proyecto y nos ubicamos en ella en la terminal:

```bash
mkdir project-backend
cd project-backend
```

Para crear el backend del proyecto con NestJS ejecutamos el siguiente comando:

```bash
nest new project-backend
```

Este creará automáticamente toda la estructura inicial del proyecto y después nos preguntará el gestor de paquetes que se deseamos utilizar. Seleccionaremos npm.

```
PS C:\Users\tomas> nest new .
✨  We will scaffold your app in a few seconds..

? Which package manager would you ❤️  to use?
> npm
  yarn
  pnpm
```

3. Levantamiento de la base de datos PostgreSQL con Docker

Para el levantamiento de la base de datos desde Docker primero se creará el contenedor con el siguiente comando desde la terminal:

```bash
docker run --name nombre_contenedor -e POSTGRES_PASSWORD=tu_contraseña -p 5432:5432 -d postgres
```

El código es bastante autodescriptivo, sin embargo, vale la pena aclarar que en el comando se define el nombre del contenedor, la contraseña del usuario postgres y se establece la utilización del puerto 5432 para el contenedor. Al darle ejecutarlo, se crea el contenedor ya será visible desde Docker desktop, con la posibilidad de ejecutar la base de datos en segundo plano con tan solo un click.

La dejaremos corriendo mientras hacemos el resto del proceso.

4. Inicialización de Prisma

Ahora se inicializa Prisma, desde la ubicación del proyecto en terminal, para configurar la conexión entre el backend desarrollado con NestJS y la base de datos PostgreSQL, utilizando el siguiente comando:

```bash
npx prisma init
```

Para este punto el directorio del Backend tu proyecto debería quedar de la siguiente manera (con los “.git” y “.gitignore” si ya lo subiste a GitHub):

Ordenar Ver ...

Nombre	Fecha de modificación	Tipo	Tamaño
.git	13/05/2026 10:49 p. m.	Carpeta de archivos	
dist	13/05/2026 7:32 p. m.	Carpeta de archivos	
generated	11/05/2026 4:37 p. m.	Carpeta de archivos	
node_modules	13/05/2026 6:54 p. m.	Carpeta de archivos	
prisma	13/05/2026 6:33 p. m.	Carpeta de archivos	
src	11/05/2026 5:51 p. m.	Carpeta de archivos	
test	11/05/2026 11:32 a. m.	Carpeta de archivos	
.gitignore	11/05/2026 11:34 a. m.	Archivo de origen ...	1 KB
eslint.config.mjs	11/05/2026 11:32 a. m.	Archivo de origen ...	1 KB
nest-cli.json	11/05/2026 11:32 a. m.	Archivo de origen ...	1 KB
package.json	13/05/2026 6:54 p. m.	Archivo de origen ...	3 KB
package-lock.json	13/05/2026 6:54 p. m.	Archivo de origen ...	404 KB
tsconfig.build.json	11/05/2026 11:32 a. m.	Archivo de origen ...	1 KB
tsconfig.json	11/05/2026 11:32 a. m.	Archivo de origen ...	1 KB
.env	11/05/2026 11:56 a. m.	Archivo ENV	1 KB
README.md	11/05/2026 11:32 a. m.	Archivo MD	6 KB
.prettierrc	11/05/2026 11:32 a. m.	Archivo PRETTIER...	1 KB
prisma.config.ts	11/05/2026 11:51 a. m.	Archivo TS	1 KB

Estos archivos son creados y generados por las instalaciones e inicializaciones ejecutadas anteriormente.

5. Configuración de variables de entorno

Prisma utiliza la siguiente variable dentro del archivo ".env" para conectarse automáticamente a la base de datos:

```env
DATABASE_URL="postgresql://postgres:contraseña@localhost:5432/postgres"
```

Prisma se encuentra dentro del backend de NestJS como una librería instalada con npm, por lo que está es la variable de entorno mínima para realizar la conexión entre la base de datos y el backend.

Posteriormente el archivo "prisma.config.ts" se configura de la siguiente manera:

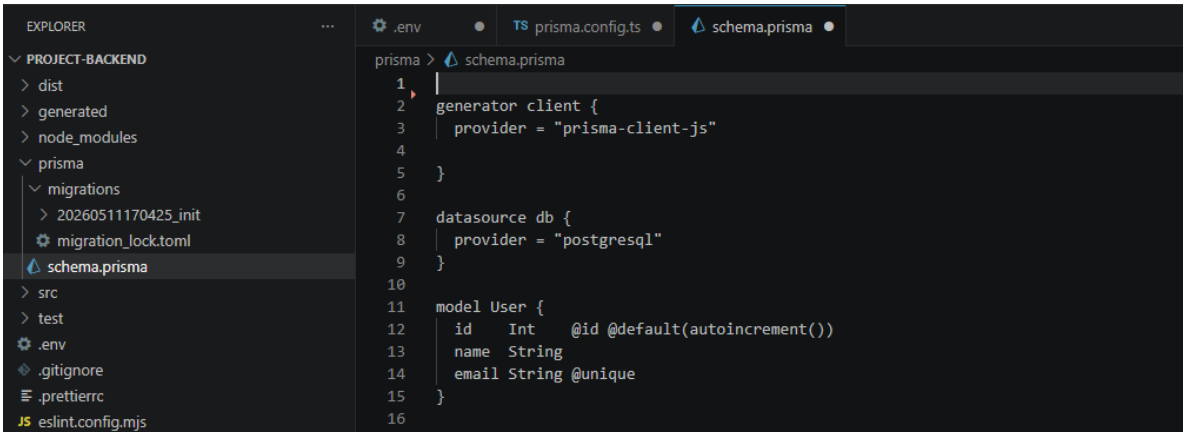
```typescript
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
  },
  datasource: {
    url: process.env["DATABASE_URL"],
  },
});
```

De esta forma se le indica a Prisma la URL definida en el archivo ".env" para conectarse a PostgreSQL, permitiéndole crear tablas, hacer migraciones y consultar información.

6. Definición y creación de entidades mínimas

Se definirá la primera entidad o tabla de la aplicación dentro de Prisma, editando el archivo “schema.prisma” dentro de la capeta prisma, de forma que quede algo así:



Una vez guardado el archivo, se creará la tabla en postgres con el siguiente comando desde la dirección del proyecto en la terminal:

```bash
npx prisma migrate dev --name init
```

Si todo funciona correctamente te aparecerá un mensaje como el siguiente:

```
Your database is now in sync with your schema.
```

Posteriormente se generará el cliente ORM de Prisma utilizando el comando:

```bash
npx prisma generate
```

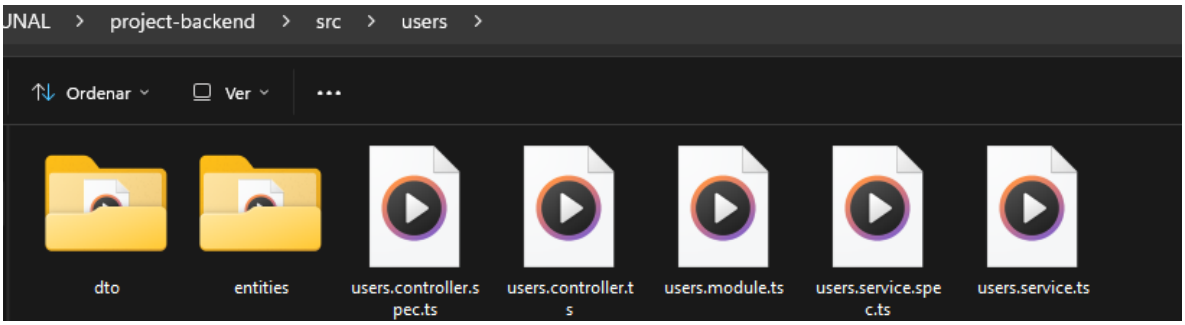
Por ultimo se utiliza el siguiente comando para generar un recurso REST, de forma que
se crea un módulo CRUD completo para la entidad Users:

```bash
nest g resource users
```

Se elige posteriormente:

```
REST API
Yes (CRUD)
```

Lo que generará los siguientes archivos, así como endpoints REST básicos:



http

```http
GET /users
POST /users
PATCH /users/:id
DELETE /users/:id
```

7. Levantamiento del servidor

Una vez completado todo lo anterior, se procede a levantar el servidor con el siguiente código en una terminal con la dirección del proyecto:

```bash
npm run start:dev
```

```
PS C:\Users\tomas\OneDrive\Documentos\UNAL\project-backend> npm run start:dev
```

Si no hay ningún error ni dependencia por descargar debería salirte lo siguiente en la terminal en la que ejecutaste este último código:

```
[10:04:22 a. m.] File change detected. Starting incremental compilation...

[10:04:23 a. m.] Found 0 errors. Watching for file changes.

[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [NestFactory] Starting Nest application...
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [InstanceLoader] PrismaModule dependencies
 initialized +10ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [InstanceLoader] AppModule dependencies in
itialized +1ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [InstanceLoader] UsersModule dependencies
initialized +0ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RoutesResolver] AppController {/}: +4ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RouterExplorer] Mapped {/, GET} route +2m
s
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RoutesResolver] UsersController {/users}:
 +1ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RouterExplorer] Mapped {/users, POST} rou
te +0ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RouterExplorer] Mapped {/users, GET} rout
e +1ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RouterExplorer] Mapped {/users/:id, GET}
route +1ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RouterExplorer] Mapped {/users/:id, PATCH
} route +0ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [RouterExplorer] Mapped {/users/:id, DELET
E} route +1ms
[Nest] 7564  - 14/05/2026, 10:04:24 a. m.     LOG [NestApplication] Nest application success
fully started +78ms
```

8. Archivo .yml

Finalmente, para facilitar la configuración y ejecución de la base de datos del proyecto, se creará un archivo ".yml" de Docker que permita levantar automáticamente una instancia de PostgreSQL mediante Docker Compose.

docker-compose.yml

C: > Users > tomas > OneDrive > Documentos > UNAL > project-backend > docker-compose.yml

```yaml
version: '3.9'

services:
  db:
    image: postgres:18
    environment:
      POSTGRES_USER: monastery
      POSTGRES_PASSWORD: 123456789
      POSTGRES_DB: monastery
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

9. Prueba

El servidor por defecto quedara disponible en la siguiente dirección: http://localhost:3000

La prueba de la API se realizó usando Thunder Client dentro de Visual Studio Code (una extensión de VSCode), enviando solicitudes HTTP al backend ejecutado con NestJS, para ello se utilizó la solicitud Post con el siguiente contenido en el apartado "body" con formato JSON:

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

POST localhost:3000/users
just now

TC New Request

POST http://localhost:3000/users Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "Tomas",
  "email": "tomas1@test.com"
}
```

Status: 201 Created Size: 49 Bytes Time: 35 ms

Response Headers 6 Cookies Results Docs </>


```
1  {
2    "id": 3,
3    "name": "Tomas",
4    "email": "tomas1@test.com"
5  }
```


Como se puede ver la solicitud fue positiva y el usuario creado.